An Automated Raspberry Pi–Based Pressure Control and Data Acquisition System for Membrane Permeability Characterization

Salvador Adrián Martínez García
ENLACE Research Program, University of California San Diego, La Jolla, CA
July 2026

Abstract

Membrane permeability characterization by constant-pressure permeate collection is conventionally performed manually: an operator regulates the feed pressure with a hand valve while simultaneously timing the collection of permeate with a stopwatch and repositioning the outlet hose between containers. Both tasks introduce operator-dependent error into the flow-rate measurement from which the Darcy permeability is derived. This work presents a low-cost automated test platform, built around a Raspberry Pi 4 single-board computer, that closes the loop on both error sources. Feed pressure is regulated at 20 Hz by a discrete proportional–integral–derivative (PID) controller — with derivative-on-measurement filtering, back-calculation anti-windup, and rate-limited setpoint ramping — commanding a servo-actuated quarter-turn valve on the compressed-air inlet of an air-over-water pressure vessel. A three-way solenoid diverter automates the timed routing of permeate between waste and measurement containers, so the collection interval is defined by the same clock that samples the pressure. Water temperature is measured in situ and the temperature-dependent viscosity is computed from the Vogel correlation, making the derived permeability temperature-corrected per run. The full software stack — hardware abstraction, control loop, playlist-based test sequencing with a manual gate between runs, safety supervision, data logging, and a browser-based user interface — was validated in a hardware-free simulation mode built on a first-order plant model. Against a ±8 kPa oscillating supply disturbance, the closed loop held setpoints of 20/40/60 kPa with standard deviations of 0.21–0.69 kPa and 100% of samples inside the ±10% acceptance band. The derived permeability was invariant with water temperature to five significant figures ($k = 1.4392 \times 10^{-12}$ m² at 20, 25, 30, and 35 °C) while the raw slope changed by 39%, as theory requires, and the analysis pipeline reproduced a reference manually acquired dataset ($k = 1.44 \times 10^{-12}$ m², mean hydraulic pore size 6.78 µm, $R^2 = 0.994$) to within reading precision.

Keywords: membrane permeability; Darcy's law; process automation; Raspberry Pi; PID control; instrumentation

1. Introduction

1.1 The measurement and why it matters

The permeability of porous membranes is routinely characterized by driving water through a membrane specimen at a series of controlled transmembrane pressures and measuring the resulting volumetric flow rate. Under laminar, viscous-dominated flow the two quantities are proportional (Darcy's law [1]), and the slope of the flow-rate–versus–pressure line yields the permeability, from which a mean hydraulic pore size follows. In the target application — woven-mesh membranes for two-phase cooling devices, where pore size controls the trade-off between permeate flow and capillary pumping pressure — permeability values in the 10^{-13}–10^{-12} m² range must be resolved reliably and repeatably across specimens, because design decisions ride on comparisons between meshes whose permeabilities differ by far less than an order of magnitude.

1.2 Anatomy of the manual procedure and its errors

In the existing manual procedure, a compressor pressurizes a water reservoir; an operator throttles a valve by hand until an analog gauge reads the target pressure, holds it there by continuous adjustment, and simultaneously starts a stopwatch while moving the outlet hose into a graduated container. After a fixed time the hose is withdrawn and the collected volume is read. Every quantity in the final result passes through a human in real time, and each pass costs accuracy in a different way:

- Pressure holding. The compressor cycles, so the supply pressure is not constant; the operator chases it with the valve while also watching the clock. At the lowest test point (20 kPa) a hand-held wander of only ±3 kPa is a ±15% error in the independent variable — and, crucially, it is an unrecorded error: the analysis is later performed against the nominal setpoint, as if the pressure had actually been held there.
- Collection timing. The interval is bounded by two manual hose transfers synchronized by eye with a stopwatch. With a human reaction time of a few tenths of a second at each end, a 60 s window carries an uncertainty of roughly 1–2% — and the permeate spilled or missed during each transfer adds a volume error of the same sign every time, i.e., a bias rather than noise.
- Attention splitting. Both tasks peak at the same moment (the start of collection), so errors in one are correlated with errors in the other. A pressure excursion during the collection window goes unnoticed precisely when it matters most.

The design insight of this work is that these two error sources call for two different remedies. Timing error is eliminated by giving the clock to the computer: a solenoid diverter switched by the same 20 Hz loop that samples the pressure bounds the collection window to software precision (one control tick, 50 ms, or <0.1% of a 60 s window). Pressure error, by contrast, is not so much eliminated as made honest: the controller holds the setpoint as well as a hobby-grade actuator allows, but — more importantly — the analysis is performed against the measured mean pressure of each collection window rather than the nominal target. Imperfect regulation then adds scatter, which the regression averages out, instead of bias, which it cannot (Section 6).

1.3 Design constraints

The platform was designed under three constraints. First, minimal cost: commodity hobby-grade actuation and sensing (a total bill of materials near US$300), accepting coarser regulation in exchange and compensating in the analysis. Second, non-invasive integration: the existing stainless-steel test vessel, its plumbing, and its analog gauge remain untouched; the automation attaches around them. Third, hardware-free verifiability: every line of control logic and analysis had to be exercisable end-to-end on a laptop, with no rig attached, so that the software could be proven before a single part was purchased — and can be regression-tested after every future change.

2. Theory of Operation

2.1 Darcy's law and the slope method

For laminar flow of an incompressible fluid through a thin porous layer, Darcy's law relates the volumetric flow rate Q (m³/s) to the transmembrane pressure difference ΔP (Pa):

$$Q = k \cdot A \cdot \Delta P / (\mu \cdot L) \tag{1}$$

where k (m²) is the permeability — a purely geometric property of the porous structure — A (m²) the active membrane area, L (m) its thickness, and μ (Pa·s) the dynamic viscosity of the fluid. Equation (1) predicts that a plot of Q against ΔP is a straight line through the origin. In practice the measured line is fitted with a free intercept,

$$Q = a + b \cdot \Delta P \tag{2}$$

and the permeability is extracted from the fitted slope b alone:

$$k = b_{\mathrm{Pa}} \cdot \mu \cdot L / A \tag{3}$$

with b_{Pa} the slope converted to SI pressure units. This *slope method* is deliberately preferred over the seemingly simpler alternative of computing k from each (ΔP, Q) point individually and averaging. The free intercept a absorbs any systematic offset common to all points — a zero error in the pressure chain, a residual hydrostatic head, a small leak — which in per-point averaging would contaminate every k estimate, and worst at the lowest pressures where the relative offset is largest. The slope, being a difference quantity, is immune to it. The coefficient of determination R^2 of the fit doubles as a built-in physical check: if the specimen truly follows Darcy's law the points must be collinear, and the software flags any fit with $R^2 < 0.98$ for inspection rather than silently reporting a permeability from data that do not support the model.

2.2 From permeability to an equivalent pore size

To convert k into an intuitive length scale, the membrane is modeled as a bundle of parallel cylindrical capillaries. Hagen–Poiseuille flow through a capillary of diameter d gives an equivalent Darcy permeability of $d^2/32$ [5]; inverting, and absorbing porosity into the definition, the laboratory reports a *mean hydraulic pore diameter*

$$d_{\mathrm{h}} = \sqrt{(32 \cdot k)} \tag{4}$$

This is an effective, model-based diameter — the size of uniform capillaries that would produce the observed permeability — not a direct geometric measurement; its value is in comparing specimens on a common scale.

2.3 Temperature, viscosity, and the invariance of k

Viscosity is the only fluid property in Eq. (3), and for water it varies steeply with temperature — about 2.2% per °C near room temperature. The system therefore measures the water temperature and evaluates μ from the Vogel correlation [2], whose constants for water reproduce tabulated experimental values [3] to better than 1% over the relevant 15–35 °C range:

$$\mu(T) = 2.414 \times 10^{-5} \cdot 10^{247.8/(T - 140)} \ \text{Pa·s}, \ T \text{ in K} \tag{5}$$

(1.002 mPa·s at 20 °C; 0.890 mPa·s at 25 °C). Because the rig runs with distilled water, the pure-water correlation applies without correction. The structure of Eq. (1) then makes a sharp, testable prediction: warmer water is thinner, so at a given pressure it flows faster and the measured slope b grows as $1/\mu$; but Eq. (3) multiplies that slope by μ again, so the derived k must not change with temperature at all. Permeability is geometry, not fluid. This exact cancellation is exploited in two ways: it makes results taken on different days at different room temperatures directly comparable, and it serves as a physical consistency check of the whole pipeline — if k drifts with temperature, something in the chain (a miscalibrated sensor, a wrong viscosity, a leak) is broken. Section 7.3 verifies the cancellation numerically.

3. System Description

3.1 Apparatus

The test cell is an existing stainless-steel pressure vessel with the membrane specimen clamped at a bolted mid-plane flange. Compressed air from the laboratory gas panel pressurizes the headspace above a column of distilled water (air-over-water configuration); the pressurized water permeates the membrane and exits through an outlet port. The automation adds four elements to this vessel (Fig. 1): (i) a hobby servomotor (20 kg·cm class) coupled through a 3-D-printed 2:1 reduction to the quarter-turn ball valve on the air inlet; (ii) a ratiometric pressure transducer (0–103.4 kPa, 0.5–4.5 V) teed into the feed line adjacent to the existing analog gauge; (iii) a waterproof DS18B20 digital thermometer immersed in the permeate stream; (iv) a 12 V three-way solenoid valve on the permeate outlet that routes flow either to waste or to the graduated measurement container. An adjustable mechanical pressure-relief valve on the air side was specified as a hardware overpressure backstop but has not been fitted; Section 8.1 states plainly what that costs.

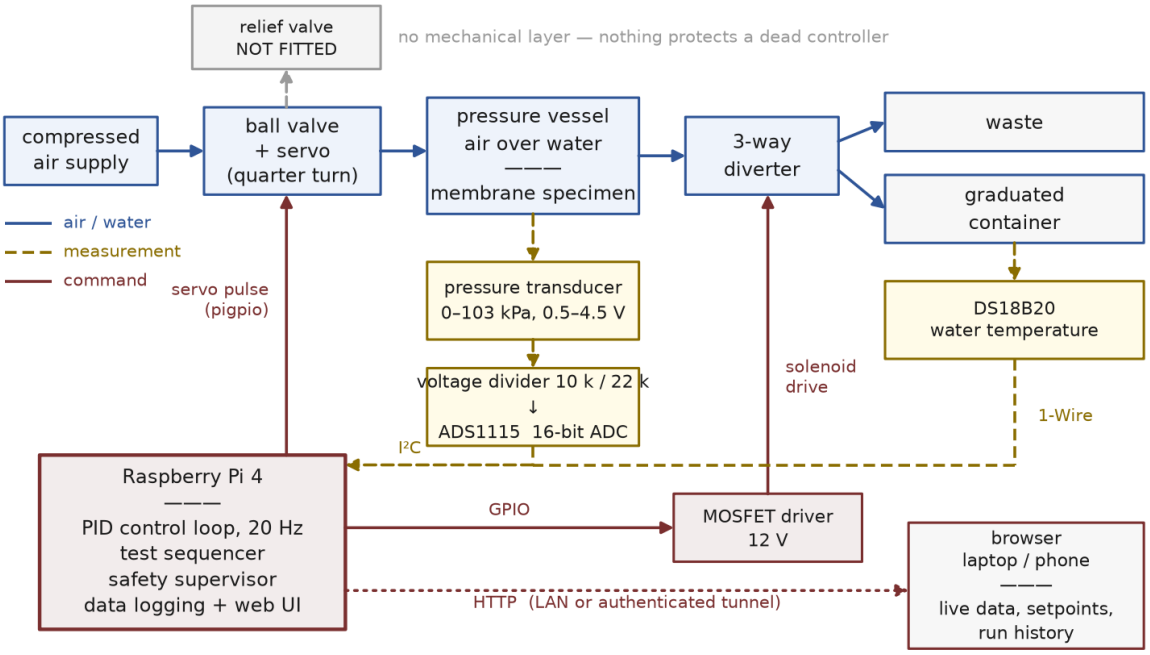

Figure 1. System block diagram. The controller closes two loops: pressure (transducer → PID → servo-actuated air valve) and collection timing (sequencer → solenoid diverter). Every protective layer shown is inside the control system: no mechanical relief valve is fitted (Section 8.1).

3.2 Sensing chain: from diaphragm to kilopascals

The Raspberry Pi has no analog inputs, so the transducer signal is digitized by a 16-bit ADS1115 analog-to-digital converter on the I²C bus, sampled at 20 Hz. Two electrical details shape the front end. First, the transducer is *ratiometric*: its 0.5–4.5 V output is a fixed fraction of its 5 V supply, so the usable signal spans 4.0 V. Second, the ADS1115 operates from the Pi's 3.3 V rail and must never see the sensor's 4.5 V directly; the signal therefore passes through a 10 kΩ/22 kΩ resistive divider with ratio 22/(10+22) = 0.6875, mapping 4.5 V down to 3.09 V. The software simply inverts the whole chain — divider, then the sensor's linear transfer function:

$$P = 103.4\ \text{kPa} \cdot (\, v_{\text{ADC}} / 0.6875 - 0.5\ \text{V}\,) / 4.0\ \text{V} \tag{6}$$

A worked example makes the chain concrete: at P = 40 kPa the transducer outputs 0.5 + (40/103.4)·4.0 = 2.047 V, the divider presents 1.408 V to the converter, and Eq. (6) recovers 40 kPa. At the converter's ±4.096 V range one count is 125 µV, which projects back through the divider and the 25.85 kPa/V sensor slope to a resolution of ≈0.005 kPa per count — two orders of magnitude finer than the control requirement, so measurement quality is limited by electrical noise (±0.1–0.3 kPa expected in practice) rather than quantization. Calibration is a two-point comparison against the existing analog gauge (atmospheric zero and one pressurized point), which also absorbs the supply-rail tolerance that ratiometric sensors inherit. Equation (6) is deliberately not clamped: an out-of-range voltage extrapolates to an implausible pressure, and it is the safety layer's job — not the driver's — to interpret that (Section 8). In the same spirit, any I²C failure makes the driver return a not-a-number reading flagged unhealthy rather than raise an exception: a sensor fault must never crash the loop that is holding a pressurized vessel.

Water temperature is read over the 1-Wire bus from the DS18B20 probe. One conversion takes ≈750 ms — fifteen control periods — so the probe is polled every 3 s from a dedicated slow thread that never blocks the control loop. Each reading updates the viscosity via Eq. (5) live during the run; the analysis then uses the run-mean temperature. If the probe fails (CRC error, missing device), the reading degrades to the configured manual temperature instead of stopping the test.

3.3 Actuation chain: from percent to valve angle

The controller expresses its output as a normalized 0–100% *pressure authority* command u: 0% is the air valve at its lowest-pressure position — the safe state, in which the vessel self-depressurizes through the membrane — and 100% fully open. The servo driver maps this linearly onto pulse width:

$$t_{\text{pulse}} = 700 + 16 \cdot u \quad \mu s \tag{7}$$

i.e., 700 µs at 0% and 2300 µs at 100%, endpoints that are calibrated to the valve's useful travel rather than the servo's mechanical limits (over-driving a stalled servo overheats it). The pulses are generated by the pigpio daemon, which times them by DMA in hardware; software-timed PWM on a multitasking operating system jitters by tens of microseconds, which on a 1600 µs span would translate into visible valve chatter. Two properties of this chain matter for control. First, a ball valve's flow characteristic is strongly nonlinear over its 90° travel, but the operating point here sits near the closed end — the flattest region — and the loop gain was tuned there. Second, and central to the design philosophy: the regulated variable of record is always the independently measured transducer pressure, never the valve position. The valve is merely how the controller pushes; the transducer is what the physics is asked about. One qualification, developed in Section 8.3: 0% is the bottom of the regulating range, not necessarily a fully sealed valve, so the end of a run drives the servo to a separate, calibrated seating position rather than leaving it at 0%.

3.4 Software architecture

The control software (Python 3) is organized in strict layers that communicate only through explicit interfaces (Fig. 2). The foundation is a hardware abstraction layer (HAL): four small abstract contracts — PressureSensor, ProportionalValve, DiverterValve, TemperatureSensor — each with two interchangeable implementations. The real drivers speak I²C, DMA-timed servo pulses, GPIO/MOSFET solenoid drive, and 1-Wire thermometry; the mock drivers bind to a first-order simulated plant (Section 7.1). A single configuration line, mode: sim, swaps one set for the other, and nothing above the HAL can tell the difference — the identical control loop, sequencer, safety supervisor, analysis pipeline, and web interface run against the simulation. This is what made it possible to validate the entire instrument before its hardware existed. Above the HAL sit the application layer (control loop, test sequencer, safety supervisor, data logger —

Sections 4, 5, 8), the analysis layer (Section 6), and the user interface layer (Section 9). Every tunable quantity — setpoints, gains, timings, limits, geometry, pin assignments — lives in one human-readable configuration file; characterizing a new membrane requires editing zero lines of code.

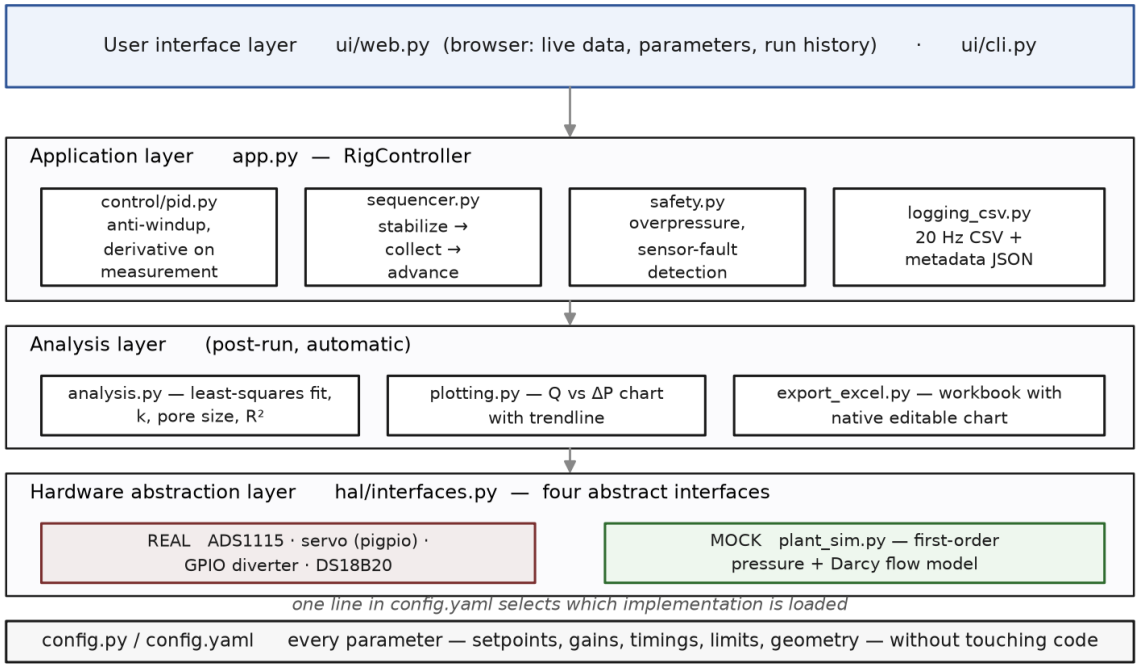

Figure 2. Software layer map. Each box is one module; arrows indicate dependency direction. The hardware abstraction layer is the only code that touches physical devices, so replacing the real drivers with the mock plant exercises everything above it unchanged.

4. The Control System

Figure 3 shows the closed loop. This section explains each element and why it is there, working from the physics of the vessel inward to the discrete control law.

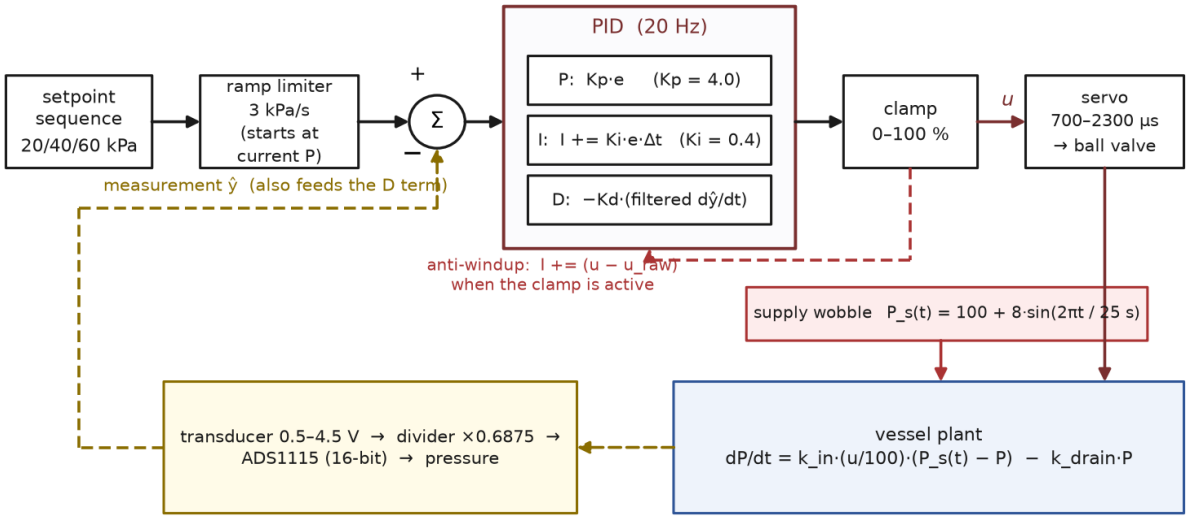

Figure 3. Closed-loop structure as implemented. The PID runs at 20 Hz; the supply wobble is the disturbance the loop must reject; the anti-windup path drains the integrator whenever the output clamp is active.

4.1 Plant dynamics: what the controller is fighting

The vessel's pressure dynamics are well approximated by a first-order balance between inflow through the air valve and outflow through the membrane:

$$dP/dt = k_{in} \cdot (u/100) \cdot (P_s(t) - P) \; - \; k_{drain} \cdot P \tag{8}$$

where $P_s(t)$ is the supply pressure and the two rate constants encode the valve's admittance and the membrane's permeation. Two features of Eq. (8) drove design decisions well beyond the controller gains. First, the plant is *asymmetric*: with the valve open the vessel fills within seconds, but with the valve closed it can only drain through the membrane, with a time constant $1/k_{drain}$ of roughly 20 s. Overshoot is therefore cheap to cause and expensive to undo — which motivates both the setpoint ramp (Section 4.3) and the sequencer's policy of visiting setpoints in ascending order, so the slow draining direction is never on the critical path. Second, the supply is not constant: compressor cycling is modeled as a sinusoidal wobble $P_s(t) = 100 + 8 \cdot \sin(2\pi t/25\ \text{s})$ kPa. This is the disturbance the loop exists to reject, and the simulation deliberately makes it worse than the bench is expected to be.

4.2 The discrete control law, term by term

Every 50 ms the loop computes the error between the (ramped) target r and the measured pressure $\hat{y}$, and updates three terms:

$$e[n] = r[n] - \hat{y}[n] \tag{9}$$

$$I[n] = I[n-1] + K_i \cdot e[n] \cdot \Delta t \tag{10}$$

$$\dot{r}[n] = \dot{r}[n-1] + \alpha \cdot \left((\hat{y}[n] - \hat{y}[n-1])/\Delta t - \dot{r}[n-1] \right), \quad \alpha = \Delta t/(\tau_d + \Delta t) \tag{11}$$

$$u[n] = clamp(K_p \cdot e[n] + I[n] - K_d \cdot \dot{r}[n],\ 0,\ 100) \tag{12}$$

with gains $K_p = 4.0$, $K_i = 0.4$, $K_d = 1.0$ and derivative filter time constant $\tau_d = 0.3$ s, all tuned in simulation against the asymmetric plant. Three implementation choices deserve explanation, because each prevents a specific failure:

- **Derivative on measurement, not on error (Eq. 11–12).** Differentiating the error would differentiate the setpoint too, so every setpoint step would fire an impulse through the D term — a "derivative kick" that slams the valve [4]. Differentiating only the measurement gives the same damping action with no kick. The sign is negative in Eq. (12): rising pressure pushes the valve toward closed.
- **A filtered derivative (Eq. 11).** The pressure reading carries ±0.15 kPa of sample-to-sample noise. Naively differencing it at 20 Hz would produce phantom rates of ±3–4 kPa/s — with $K_d = 1$, several percent of full valve authority in pure noise. The first-order filter ($\alpha \approx 0.14$ at these settings) averages the rate estimate over ≈0.3 s, trading a small lag for a usable signal.
- **Back-calculation anti-windup (the dashed path in Fig. 3).** When the demanded output exceeds the clamp — e.g. the raw sum is 112% but the valve saturates at 100% — a plain integrator would keep accumulating error it can never act on, then discharge it as a large overshoot once the error reverses. Here the excess ($100 - 112 = -12$) is added back into the integrator in the same tick, so the integrator always holds exactly the value consistent with the output actually applied. Saturation then ends cleanly, with no stored surplus to burn off.

4.3 Setpoint ramping

Setpoint changes are not applied as steps. When the sequencer moves to a new target, the controller's internal reference starts at the current measured pressure and slews toward the true setpoint at 3 kPa/s. This keeps the error term small throughout the transit — so the integrator never winds up during the approach — and makes the loop arrive at each target from below, which on the asymmetric plant of Section 4.1 suppresses overshoot and also takes up any mechanical backlash in the printed valve transmission from a consistent direction. One subtlety: the acceptance test for "stabilized" (Section 5) is always evaluated against the true setpoint, never the moving ramp target, so ramping cannot fake a stabilization.

4.4 Loop timing

The control thread runs on a fixed timebase: each tick's deadline is the previous deadline plus 50 ms, not "now plus 50 ms," so timing errors do not accumulate. If a tick overruns (e.g., the operating system stalls the thread), the scheduler re-synchronizes to the current time rather than firing a burst of catch-up ticks — a burst would feed the integrator several identical error samples in a few milliseconds, which is indistinguishable from a plant that stopped responding.

5. Automated Test Sequencing

Sequencing operates at two levels. A run drives one or more setpoints to completion unattended; a playlist chains runs together with a mandatory manual gate between them, because the operations that separate one run from the next — reading and emptying the graduated container, and often exchanging the specimen — cannot be automated and must not be skipped. This section describes the run first (Section 5.1), then the playlist that assembles a full characterization from runs (Section 5.2).

5.1 Sequencing a single run

Within a run, each test point requires: reach the pressure, prove it is being held, collect permeate for a precisely known interval, and move on. A four-state finite-state machine (IDLE → STABILIZING → COLLECTING → … → DONE) encodes this protocol; Figure 4 shows it executing on the simulated plant.

- **STABILIZING.** The PID drives toward the setpoint while the sequencer watches the measured pressure against an acceptance band of ±10% of the setpoint (±2 kPa at 20 kPa). The pressure must remain inside the band continuously for a 5 s dwell; a single excursion resets the dwell clock to zero. This continuity requirement is what distinguishes "regulating" from "passing through": a pressure that oscillates across the band can never accumulate 5 s in band, no matter how often it visits. If stabilization is not achieved within 180 s, the point is recorded as failed with an explicit reason and the sequence advances — one unreachable setpoint costs one data point, not the run.
- **COLLECTING.** On entry the solenoid diverter switches the permeate stream from waste to the graduated container, and a fresh statistics accumulator starts. For exactly 60 s (1200 samples at 20 Hz) the sequencer accumulates the pressure's mean, standard deviation, minimum, maximum, and in-band fraction — over the collection window only, because that is the only interval whose pressure history matters to the flow measurement. On exit the diverter drops back to waste (its de-energized, fail-safe position) and the machine advances to the next setpoint or to DONE.
- **Ascending order.** Setpoints are visited low-to-high. The plant of Eq. (8) fills in seconds but drains in tens of seconds; descending steps would spend most of the run waiting for pressure to leak away through the membrane.

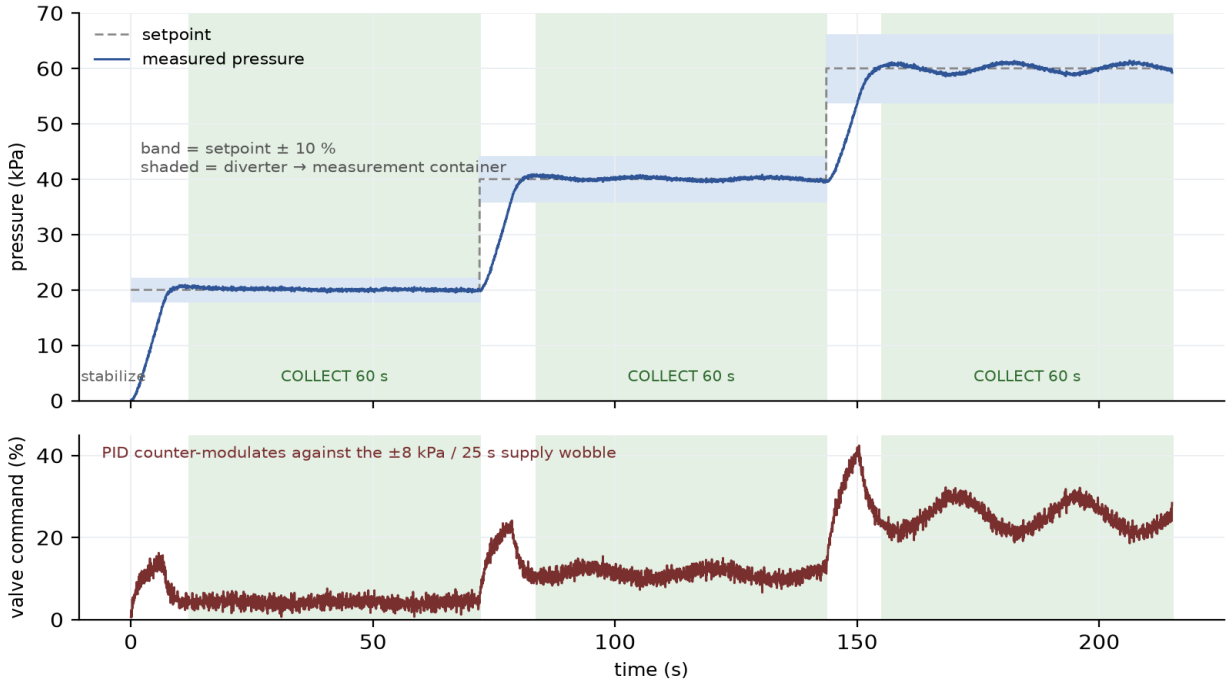

Figure 4. A complete simulated 20/40/60 kPa sequence (215 s). Top: measured pressure, setpoint, ±10% acceptance bands, and the three 60 s collection windows (shaded). Bottom: valve command. The ~25 s periodicity visible in the valve trace — most clearly at 60 kPa — is the PID counter-modulating against the ±8 kPa supply wobble; the pressure trace above it stays flat.

The volume collected in each window is read from the graduated container and entered by the operator at the end of the sequence — the single remaining manual step — and the flow rate for point i follows as

$$Q_i = V_i / t_c \,, \tag{13}$$

with the collection time t_c known to software precision. In simulation the volume is instead integrated directly from the modeled flow, which is what allows the end-to-end validation of Section 7 to run with no operator at all. A detail of the terminal state: once the sequence ends, the controller latches a sticky "finished" flag that persists until the next start command, so a user interface polling every 500 ms cannot miss the completion event between two control ticks.

5.2 Experiment playlists and the manual gate

A complete characterization is rarely a single run. It is a set of pressure points on one specimen — and, when meshes are compared, on several specimens in turn — separated by physical operations no controller can perform: the graduated container has to be read and emptied before its volume is lost, and the specimen may need to be unclamped and replaced. The software therefore organizes runs into a playlist, an ordered queue of experiments executed strictly one at a time. The defining property of the queue is that it never advances on its own. When an item finishes, the rig returns to its safe state — feed valve sealed, diverter to waste (Section 8.3) — and stops; the next item begins only when the operator presses play. Auto-advancing would either pressurize a membrane with nobody standing next to it or discard the permeate volume that is the entire measurement, so the gate serves data integrity and safety at once.

Each queued item carries its own setpoints and timing — tolerance, dwell, collection window, stabilization timeout — so different points can be measured under different conditions without changing anything global.

An item is usually a single pressure, but a multi-point item is permitted and runs its points back-to-back unattended: the gate falls between items, where the human work is, not between the pressures within one item. The specimen's pressure limit travels with the queue rather than with the configuration file, because it is a property of the mesh currently clamped in the vessel, not of the hardware; it is stored beside the playlist and edited from the interface without touching the configuration file, and like every operator-facing limit it can only tighten, never loosen (Section 8.1).

Because the points of one specimen are deliberately split across separately gated runs, the deliverable is the combined fit rather than the per-run one: when the volumes are entered, the analysis of Section 6 regresses flow against pressure over every measured point pooled from all completed items in the queue. The playlist is persisted to disk on every change, so a mid-session restart loses neither the queue nor the points already collected, and a run caught in progress by a restart is recorded as failed rather than left falsely marked running — so an interrupted point is never mistaken for a good one.

6. Automated Data Analysis

When the volumes are entered, the system fits Eq. (2) to the points $(\bar{P}_i, Q_i)$ — pooled across the completed items of the playlist (Section 5.2), excluding any run flagged as having continued under a raised ceiling (Section 8.1), where $\bar{P}_i$ is the *measured mean* pressure of collection window i, not the nominal setpoint — by ordinary least squares:

$$b = \Sigma(\bar{P}_i - \bar{P})(Q_i - \bar{Q}) \,/\, \Sigma(\bar{P}_i - \bar{P})^2 \tag{14}$$

then converts the slope to SI units ($b_{Pa} = b/1000$ for pressures logged in kPa) and applies Eqs. (3)–(4) with μ evaluated at the run-mean measured water temperature. Regressing against measured means is the analytical half of the design argument begun in Section 1.2: the regulation layer's job is only to keep each cluster of pressures near its target; the analysis never pretends the pressure was anything other than what the transducer recorded. Coarse regulation therefore widens the scatter of points along the fitted line — which R^2 reports honestly — but does not displace the line. The fit, the derived k and d_h, and the R^2 verdict are computed automatically the moment volumes are entered, and every run leaves a complete, self-describing evidence trail on disk:

- A 20 Hz time-series CSV — timestamp, phase, setpoint, measured pressure, valve command, diverter state, in-band flag, water temperature — sufficient to reconstruct the entire run.
- A metadata JSON with the per-window statistics (mean, standard deviation, minimum, maximum, in-band fraction, sample count), the entered volumes, and the configuration that produced them.
- An analysis JSON with the fit (slope, intercept, R^2), k, pore size, viscosity, and temperature.
- A publication-style chart image (Fig. 5), and a native spreadsheet workbook containing the raw table, the derived quantities, and a genuine embedded scatter chart with a linear trendline — so the spreadsheet recomputes the equation and R^2 live if a volume is corrected after the fact. Points taken under a raised ceiling are drawn and labelled rather than silently dropped — a combined fit shows them set aside, while a single-run fit, which does not consult the queue, marks them as having entered the reported k.

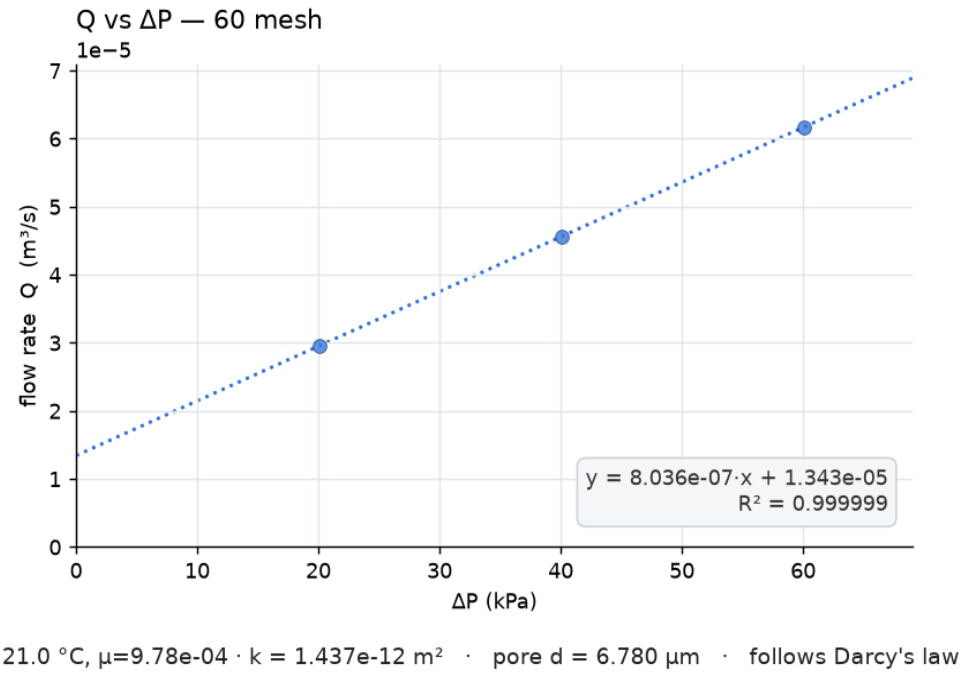

Figure 5. The chart the instrument itself produces for the simulated run of Fig. 4: flow rate versus measured mean pressure, the fitted line (Eq. 2), and the derived permeability and pore size, annotated with the water temperature and viscosity used.

7. Validation in Simulation

7.1 Method

All control logic, sequencing, safety behavior, and analysis were exercised against the mock plant of Eq. (8), configured with a 100 kPa supply, the ±8 kPa / 25 s wobble, Gaussian process noise, ±0.15 kPa sensor noise, and a Darcy-consistent permeate flow law whose magnitude scales as 1/μ with the simulated water temperature — so the plant physically embodies the viscosity dependence the analysis is supposed to cancel. The runs reported below execute the exact production code path (sequencer, PID, safety, analysis); only the clock is accelerated, which is possible precisely because every module takes time as an input rather than reading it.

7.2 Disturbance rejection

Table 1 summarizes a complete 20/40/60 kPa sequence at 21 °C under the full disturbance model (the run of Fig. 4). The closed loop held every setpoint with a standard deviation of 0.21–0.69 kPa — an order of magnitude inside the ±10% acceptance band — and 100% of collection-window samples in band, while the valve command counter-modulated by 7–14% of full scale to absorb the supply wobble. The bottom panel of Fig. 4 makes the division of labor visible: the disturbance appears in the actuator, so that it does not appear in the pressure.

Setpoint (kPa)	Mean (kPa)	Std. dev. (kPa)	Samples in band	Valve span (%FS)
20	20.09	0.21	100%	7.0
40	40.08	0.28	100%	8.9

Setpoint (kPa)	Mean (kPa)	Std. dev. (kPa)	Samples in band	Valve span (%FS)
60	60.06	0.69	100%	13.7

Table 1. Setpoint tracking under the ±8 kPa / 25 s supply disturbance (simulation, 1200 samples per window). "Valve span" is the peak-to-peak counter-modulation of the valve command during the collection window.

7.3 Temperature invariance of the derived permeability

Running the identical simulated specimen at 20, 25, 30, and 35 °C (Table 2), the raw fitted slope grew by 39% — exactly tracking 1/μ as the thinner water flowed faster — while the derived permeability remained 1.4392×10^{-12} m² to five significant figures at all four temperatures (0.000% spread). The cancellation predicted in Section 2.3 holds through the entire implemented pipeline: simulated physics, control, sequencing, statistics, viscosity model, and fit. On the physical instrument this same sweep becomes a commissioning diagnostic: a k that drifts with water temperature indicts the measurement chain, not the membrane.

T (°C)	μ (mPa·s)	Slope ((m³/s)/kPa)	k (m²)
20	1.002	7.859×10^{-7}	1.4392×10^{-12}
25	0.890	8.842×10^{-7}	1.4392×10^{-12}
30	0.797	9.875×10^{-7}	1.4392×10^{-12}
35	0.718	1.096×10^{-6}	1.4392×10^{-12}

Table 2. Temperature sweep of the identical simulated specimen: the slope varies with viscosity; the permeability does not.

7.4 Reproduction of a reference manual dataset

The analysis pipeline was benchmarked against a manually acquired 60-mesh dataset previously analyzed in a spreadsheet. The automated fit returned slope 7.858×10^{-7} (m³/s)/kPa, $R^2 = 0.9936$, $k = 1.437 \times 10^{-12}$ m², and dh = 6.78 μm, matching the spreadsheet reference (7.86×10^{-7}; 0.9936; 1.44×10^{-12}; 6.79 μm) to within reading precision. One contrast between Sections 7.2 and 7.4 is instructive: the simulated runs fit with $R^2 \approx 0.99999$ because their 2% instantaneous flow noise is averaged over 1200 samples per window, while the real manual dataset carries $R^2 = 0.9936$ — the difference is a measure of exactly the operator-dependent scatter this instrument is built to remove.

8. Safety Design

Protection is layered so that no single failure — software, sensor, or actuator — can leave the vessel pressurizing uncontrolled (Table 3). The safe state is "air inlet sealed, diverter to waste": with the feed shut, the vessel self-depressurizes through the membrane, and the diverter reaches waste by de-energizing, so it fails safe on any power loss. The servo does not share that property — it holds position when power is lost — and the mechanical relief valve that would have covered exactly that case was specified but not fitted. The consequence deserves stating without euphemism: every layer in Table 3 requires the controller to be alive. Should the single-board computer hang or lose power with the air valve open, nothing in the instrument closes it, and the pressure the vessel reaches is bounded only by what the gas panel can deliver. Two operating measures follow, neither of them a substitute: the supply regulator must be set low enough that its maximum delivery is itself a pressure the specimen tolerates — that setting has not yet been established, and the software keeps the operator ceiling-raise of Section 8.1 disabled until it is — and the

supply is closed by hand at the end of a session rather than trusting the servo to hold. Fitting the relief valve remains the highest-value hardware addition to this instrument (Section 10).

8.1 The pressure ladder

Layer	Pressure (kPa)	Action
Operating range	≤ 60	Normal tests
Specimen limit	65 (editable)	Cannot be queued or started above
Per-run ceiling	min(max(setpoint) + 10, specimen limit)	Hold at safe state, alarm
Software cutoff	80	Close valve, abort run, alarm
Mechanical relief valve	not fitted	— no layer acts without the controller
Sensor full scale	103	Saturation bound of transducer

Table 3. Defense-in-depth pressure ladder. The specimen limit and per-run ceiling are tighter than the global software cutoff and are the thresholds that actually govern a normal test.

Two of these layers sit below the 80 kPa software cutoff and are, in practice, the ones that govern a normal test. The first is the specimen limit: a per-mesh ceiling — 65 kPa for the 60-mesh specimen — below which every setpoint must fall, enforced at admission, so the sequencer refuses to queue or start a run that asks for more. It travels with the playlist (Section 5.2), is editable from the interface, and can only be tightened. The second is the per-run ceiling. Leaving the abort threshold at the global 80 kPa during a 20 kPa test would let a stuck-open valve drive the vessel to nearly four times the target before anything intervened — ruinous for a delicate mesh. So when a run starts, the supervisor tightens its cutoff to max(setpoint) + 10 kPa — itself clamped to the specimen limit and the global cutoff — for the duration of that run, relaxing it back to the global value when the run ends: a 20 kPa point is caught near 30 kPa, not 80, and a specimen limited to 65 kPa is never driven past 65, even at a top setpoint whose max(setpoint) + 10 would otherwise reach 70. The ladder is therefore not fixed — its governing rung is whatever the current specimen and the current run have asked for, and the global cutoff is only the ceiling of last resort.

Reaching that per-run ceiling does not end the run, because the ceiling is a precaution rather than a limit of the vessel. The rig stops at the safe state — feed sealed — and alarms with the setpoint, the ceiling, the pressure reached and which rung fired, then waits: the operator may retry the point, up to three times, or stop. The alarm distinguishes a plain overshoot, where retrying is reasonable, from pressure still climbing while the loop is already commanding the valve shut — the signature of a stuck valve or a lying sensor, where a retry would only repeat the excursion. A retry restarts the point from scratch and discards the partial collection, so a retried point is exactly as clean as one that never stopped. What is never recoverable is the global cutoff or a sensor or plant fault: both vent and end the run, and both stay armed while the rig is held, so a genuine runaway can never sit waiting behind a dialog for a human. A third option — raising the ceiling and continuing — exists but is disabled by configuration until the pressure available at the gas panel is known, since with nothing above it in software the supply is the only bound on a runaway. Were it enabled, any run that continued under a raised ceiling is flagged as such and excluded by default from the combined fit of Section 6: it saw pressure outside the envelope its specimen was declared to tolerate, so its permeability is not comparable with the rest.

8.2 The safety supervisor

The safety supervisor runs on every control tick, before the controller, and independent of test state. Its logic distinguishes two failure classes with different urgencies. Overpressure — a healthy, numerical reading above the effective cutoff, which is the tightened per-run ceiling of Section 8.1 while a test runs and the global 80 kPa when idle — acts on the very first sample: close and alarm, ending the run outright at the global cutoff but holding for the operator at a run ceiling (Section 8.1). Sensor faults use the opposite policy. A reading is classified as bad if the driver flagged it unhealthy, if it is not a number, or if it falls outside the physically plausible −5 to 105 kPa window; only three consecutive bad reads declare a fault, so a single I²C glitch does not abort a one-hour session, while a real failure is caught within 150 ms. During those grace ticks the PID does not act on the bad number: a non-finite reading makes it hold its last command and leave the integrator untouched, so one stray NaN cannot latch the valve fully open before the reading recovers or the supervisor vents. The plausibility window exists because of a specific, dangerous failure mode identified during design review: a disconnected transducer reads 0 V, which Eq. (6) maps to −12.9 kPa. A controller that believed that number would see "pressure far too low" and open the air valve wide against a dead sensor. Because −12.9 kPa is below the −5 kPa plausibility floor, the supervisor instead declares an instrument fault and forces the safe state.

The plausibility floor, though, only catches a sensor stuck at an implausible value; one frozen at a plausible reading would lie with a credible number the supervisor would believe, while the PID opened the valve wider against a cell that may already have climbed past the target. Two further detectors therefore ask not whether the number is credible but whether the instrument still works. The first watches liveness: a live transducer signal always carries a count or two of electrical noise, so a raw ADC value bit-identical for ten consecutive samples (0.5 s) is physically impossible from a healthy sensor and is faulted — a dead transducer cannot fake noise, which is exactly the stuck-at-a-plausible-value case the credibility checks miss. The second watches authority: if the valve is pinned open (≥ 70%) for eight seconds while the pressure has not risen even 2 kPa from where it sat when the valve pinned, the loop has lost its grip on the plant — a stuck valve, a closed supply, or a sensor reading low while the cell really climbs — and the run vent-aborts; the test is integrated rather than a rate, so a legitimate slow approach to a hard setpoint, which does rise before it flattens, is left to the stabilization timeout instead.

Two further guarantees close the remaining gaps: the entire control tick runs inside an exception handler whose failure path is the safe state (a software bug vents the vessel rather than abandoning it mid-command), and the commissioning checklist includes a live kill test — unplugging the transducer mid-run must vent and abort within the grace window.

8.3 Sealing the valve and verifying closure

The safe state names an air inlet that is sealed, but sealing it is less trivial than commanding 0%. The 0% output is the bottom of the control range — where regulation stops — and with backlash in the printed servo-to-valve coupling that position can leave the ball valve slightly cracked. A specimen left under a cracked feed valve keeps seeing supply pressure with nobody watching. So ending a run, or aborting on a fault, does not simply command 0%: it invokes a distinct full-close action that drives the servo to a separately calibrated seating pulse — set past the 0% endpoint, at the point where flow actually stops — and holds it there for a fixed interval before releasing the servo, so the stem reaches the seat instead of being abandoned mid-travel.

Sealing is then verified rather than assumed. With the feed genuinely shut, the vessel must begin to depressurize through the membrane; if it does not, the valve did not seat. After each run the supervisor

records the pressure at closure and, a fixed interval later (20 s), checks that it has fallen by at least a small margin (1 kPa). If it has not — the signature of a valve still feeding the vessel — the interface raises "valve may not have closed," directing the operator to inspect the valve and shut the supply by hand rather than silently leaving a pressurized specimen behind; the check is skipped only when the vessel was already near atmospheric at closure, where there is nothing to verify. This closes the one gap the mechanical relief valve cannot: the relief guards against pressure that is too high, while the closure check guards against a feed that never actually stopped.

9. Remote Operation and Data Management

The Pi serves a self-contained single-page web interface — no external assets, so it works on an isolated lab network — through which any browser can watch the live pressure and temperature (polled every 500 ms), edit every test parameter (setpoints, tolerance, dwell, collection time, PID gains) without code changes, start and stop sequences, enter the measured volumes, and browse the accumulated history of runs, each downloadable as chart, spreadsheet, raw CSV, or JSON. The HTTP surface is a small typed API (status, start, stop, analyze, runs) that a future script or laboratory information system could drive directly. Two deployment details matter for a device that actuates hardware. The server binds only to the loopback interface; off-network access is provided by an outbound authenticated tunnel to a subdomain, which traverses the university firewall without any inbound port and places an identity check (email allow-list) in front of every request — the interface that can open an air valve is never exposed bare. And every run file is served through a strict name filter (timestamps only), so the download endpoints cannot be used to walk the filesystem.

10. Limitations and Future Work

- Coarse pressure regulation. A servo-driven ball valve is expected to regulate to roughly ±10–15% on the physical rig, versus the ±2% of a laboratory proportional valve. This is an accepted trade, made safe by the architecture: the analysis regresses against measured window means (Section 6), so regulation coarseness widens scatter but does not bias k. A stepper-driven needle valve is the identified upgrade path if finer holding proves necessary.
- Simulation-based evidence. All quantitative results in Section 7 are simulation-based. The plant model is first-order and its constants are estimates; the real vessel will differ, and the PID gains will be retuned against it. What the simulation does establish is the correctness of the logic — sequencing, safety, statistics, analysis — which is exactly the part that does not change when the plant does.
- No hardware-only protective layer. The specified mechanical relief valve was not fitted, so every layer of Table 3 depends on the controller running (Section 8). Until it is added, the supply regulator setting and the end-of-session manual shut-off are the only bounds that survive a dead controller — and fitting it is the cheapest, highest-value change available to this instrument.
- Actuator torque margin. The valve-stem breakaway torque has not yet been measured; the printed transmission includes a 2:1 reduction as margin, and a higher-torque servo is the fallback.
- Manual volume reading. Permeate volume is still read by eye from the graduated container; an electronic scale streaming into the existing volume-entry API would close the last manual step and, incidentally, provide continuous flow curves rather than window averages.

- Feed-forward. If compressor cycling on the real rig exceeds what feedback alone rejects, a second transducer on the supply side (the ADC has three spare channels) enables feed-forward compensation: the controller would see the supply dip before the vessel does.

11. Conclusion

A complete automation platform for constant-pressure membrane permeability testing was designed, implemented, and validated in simulation. The two operator-dependent elements of the manual protocol — pressure holding and collection timing — are replaced by a 20 Hz PID loop whose implementation choices (derivative-on-measurement, filtered rate, back-calculation anti-windup, ramped targets) each neutralize a specific practical failure, and by solenoid-switched routing timed by the same clock that samples the pressure. The analysis regresses flow against measured mean pressures, converting residual regulation error from hidden bias into visible scatter, and corrects for water viscosity using in-situ temperature so that the reported permeability is a property of the membrane alone — a claim the system itself verifies, holding k constant to five significant figures across a 15 °C temperature sweep while the raw slope changed by 39%. A full multi-pressure characterization now runs as a gated playlist — one play press and one volume reading per point, with the rig sealing its feed and waiting in the safe state between points so the operator can read and reset the collection — and yields the fitted permeability, pore size, chart, and spreadsheet automatically from the pooled data. The layered architecture, with its swappable simulated plant, allowed every one of these claims to be tested before the hardware was purchased, and the same simulation now stands as a permanent regression harness for the instrument's future development.

Acknowledgments

The author thanks the TEMP Laboratory at the University of California San Diego and the ENLACE research program for the experimental context and the membrane characterization methodology on which this instrument is based.

References

[1] H. Darcy, Les fontaines publiques de la ville de Dijon, Victor Dalmont, Paris, 1856.

[2] H. Vogel, “Das Temperaturabhängigkeitsgesetz der Viskosität von Flüssigkeiten,” Physikalische Zeitschrift, vol. 22, pp. 645–646, 1921.

[3] L. Korson, W. Drost-Hansen, and F. J. Millero, “Viscosity of water at various temperatures,” J. Phys. Chem., vol. 73, no. 1, pp. 34–39, 1969.

[4] K. J. Åström and T. Hägglund, Advanced PID Control, ISA — The Instrumentation, Systems, and Automation Society, 2006.

[5] F. M. White, Fluid Mechanics, 8th ed., McGraw-Hill Education, New York, 2016.

Software availability: the full control software, simulation mode, assembly documentation, and the scripts that regenerate every figure and number in this paper are maintained in a version-controlled repository available from the author.